THE LIST AI)T:

List is an ordered set of elements.

The general form of the list is $A_1, A_2, \ldots\ldots, A_N$

A_1 - First element of the list

A_2- 1 st element of the list

N –Size of the list

If the element at position i is A_i, then its successor is A_{i+1} and its predecessor is A_{i-1}**Various operations performed on List**

1. Insert (X, 5)- Insert the element X after the position 5.

2. Delete (X) - The element X is deleted

3. Find (X) - Returns the position of X.

4. Next (i) - Returns the position of its successor element i+1.

5. Previous (i) Returns the position of its predecessor i-1.

6. Print list - Contents of the list is displayed.

7. Makeempty- Makes the list empty.

Implementation of list ADT:

1. Array based Implementation

2. Linked List based implementation

Array Implementation of list:

Array is a collection of specific number of same type of data stored in consecutive memory

locations. Array is a static data structure i.e., the memory should be allocated in advance and the

size is fixed. This will waste the memory space when used space is less than the allocated space.

Insertion and Deletion operation are expensive as it requires more data movements

Find and Print list operations takes constant time.

The basic operations performed on a list of elements are

a. Creation of List.

b. Insertion of data in the List

c. Deletion of data from the List

d. Display all data‟s in the List

e. Searching for a data in the list

Declaration of Array:

#define maxsize 10

int list[maxsize], n ;

```
#include<stdio.h>
#define MAX 10

void create();
void insert();
void deletion();
void search();
void display();
int a,b[MAX], n, p, e, f, i, pos;

void main()
{

int ch;

while(1)
{
printf("\n main Menu");
printf("\n 1.Create \n 2.Delete \n 3.Search \n 4.Insert \n 5.Display\n 6.Exit
\n");
printf("\n Enter your Choice");
scanf("%d", &ch);
```

```
switch(ch)
{
case 1:
create();
break;
case 2:
deletion();
break;
case 3:
search();
break;
case 4:
insert();
break;
case 5:
display();
break;
case 6:
exit();
break;
default:
printf("\n Enter the correct choice:");
}
}
}
void create()
{
printf("\n Enter the number of nodes");
scanf("%d", &n);
for(i=0;i<n;i++)
{
printf("\n Enter the  Element:",i+1);
scanf("%d", &b[i]);
}
}
```

```
void deletion()
{
printf("\n Enter the position u want to delete::");
scanf("%d", &pos);
if(pos>=n)
{
printf("\n Invalid Location::");
}
else
{
for(i=pos;i<n;i++)
{
b[i-1]=b[i];
}
n--;
}
printf("\n The Elements after deletion");
for(i=0;i<n;i++)
{
printf("\t%d", b[i]);
}
}


void search()
{
printf("\n Enter the Element to be searched:");
scanf("%d", &e);

for(i=0;i<n;i++)
{
if(b[i]==e)
{
printf("Value is in the %d Position", i);
}
}
```

```
if (i==n)
printf("element not found");
}


void insert()
{
printf("\n Enter the position u need to insert::");
scanf("%d", &pos);

if(pos>=n)
 {
 printf("\n invalid Location::");
 }
 else
 {
 for(i=n-1;i>=pos-1;i--)
 {
 b[i+1]=b[i];
 }
 printf("\n Enter the element to insert::\n");
 scanf("%d",&p);
 b[pos-1]=p;
 n++;
 }
 printf("\n The list after insertion::\n");

display();
}


void display()
{
printf("\n The Elements of The list ADT are:");
for(i=0;i<n;i++)
{
printf("\n\n%d", b[i]);
```

```
}
}
```